CloudWatch AI

Implementation Report

Real-Time AIOps Platform for Predictive Incident Detection

Course: CS6905 – Cloud Information Management Systems

Instructor: Dr. Shadi Aljendi

University of New Brunswick

Team Members: Alfarizy Alfarizy (3810253)
Akinbobola Akin (3784664)
Ishimwe Pacis Hanyurwimfura (3787234)

Date: March 30, 2026

Repository: https://github.com/helloimrizy-pages/cloud-project

Live Dashboard: http://cloud-project-dashboard1.s3-website-us-east-1.amazonaws.com

API Endpoint: https://p9fpx4nhh6.execute-api.ca-central-1.amazonaws.com

Contents

1 Executive Summary **3**

2 Architecture Implementation **3**
2.1 Architecture Overview . . . 3
2.2 Cloud Infrastructure Components . . . 4
2.3 Lambda Functions . . . 4
2.4 S3 Bucket Configuration . . . 5

3 Database Setup **6**
3.1 DynamoDB Table Design . . . 6
3.2 Per-User Data Isolation . . . 6
3.3 Key Design Choices . . . 7
3.4 Variable Schema per Service . . . 7
3.5 Sample Data . . . 7

4 API and Backend Development **8**
4.1 API Gateway Configuration . . . 8
4.2 Authorization Verification . . . 9
4.3 Simulation Scenarios . . . 9
4.4 Data Pipeline (Event-Driven) . . . 10
4.5 Inference Lambda Implementation . . . 10
4.6 Sample API Responses . . . 11

5 Frontend Integration **12**
5.1 Technology Stack . . . 12
5.2 Application Pages . . . 12
5.2.1 Page 1: Live Controls (`/`) . . . 12
5.2.2 Page 2: KPI Timeline (`/timeline`) . . . 12
5.2.3 Page 3: Alerts & Incidents (`/alerts`) . . . 13
5.2.4 Page 4: Analytics (`/analytics`) . . . 13
5.3 Authentication Flow . . . 13
5.4 Theme System . . . 14
5.5 API Integration . . . 14

6 Security Configuration **18**
6.1 Authentication . . . 18
6.2 Authorization – JWT Authorizer . . . 18
6.3 Data Encryption . . . 19
6.4 Access Controls . . . 19
6.5 OWASP Top 10 Security Audit . . . 19

7 Integration Testing **21**
7.1 Testing Methodology . . . 21
7.2 Test Environment . . . 21
7.3 Test Results Summary . . . 22
7.4 Key Test Cases and Results . . . 22
7.4.1 Authentication Tests (5/5 Passed) . . . 22

7.4.2 API Endpoint Tests (14/14 Passed) . . . 22
7.4.3 Simulation Lifecycle Tests (5/5 Passed) . . . 23
7.4.4 Database Verification Tests (5/5 Passed) . . . 23
7.4.5 Production Deployment Tests (3/3 Passed) . . . 23
7.5 Manual Test Documentation . . . 24

8 Challenges and Solutions **24**

9 ML Model Performance **26**
9.1 Model Configuration . . . 26
9.2 Feature Engineering . . . 26
9.3 Performance Results . . . 27

10 Performance Testing **27**
10.1 Lambda Execution Latency . . . 28
10.2 Cold Start Analysis . . . 28
10.3 End-to-End Pipeline Latency . . . 28
10.4 Memory Utilization . . . 29
10.5 Dashboard Polling Performance . . . 29

11 Deployment **29**
11.1 Infrastructure . . . 29
11.2 Cost . . . 30
11.3 Frontend Deployment . . . 30
11.4 Model Deployment . . . 31

12 Functionality Demonstration Video **31**

13 Conclusion **31**

1 Executive Summary

CloudWatch AI is a serverless AIOps platform we built to predict cloud service incidents before they actually occur. We designed and deployed the entire solution on AWS, training Gradient Boosting machine learning models on 2.4 million timesteps of cloud service KPI data. In practice, the system delivers early warnings at **0.91 precision** with a **∼36-minute average lead time**, and it produces fewer than one false alarm per service per day.

Key Achievements

- **13 Lambda functions** that handle everything from data ingestion to the REST API
- **6 DynamoDB tables** with Streams configured for event-driven processing
- **19 JWT-protected API routes** served through API Gateway
- **5 monitored cloud services** that are running real-time simulations across **6 scenarios**
- **3 ML models**, one for each prediction horizon (5, 10, and 15 minutes)
- **Per-user data isolation** through Cognito-based `owner_id` partitioning on every table
- **Email alert notifications** via Amazon SNS that is filtered on a per-user basis
- **React 19 dashboard** with 4 interactive pages, auto-polling, and a dark/light theme toggle
- **AWS Cognito authentication** using the SRP protocol with email verification
- **OWASP Top 10 audit**: 9 out of 10 passed, 1 not applicable (overall risk: Low)
- **50+ integration test cases** for all system layers

Taken together, the platform shows what a complete, production-grade cloud database application looks like end to end, from data generation and ML inference all the way through to real-time visualization and security hardening. Every component runs serverlessly within the AWS Free Tier in the `ca-central-1` region.

2 Architecture Implementation

2.1 Architecture Overview

We structured CloudWatch AI around a **serverless, event-driven architecture** on AWS. By relying entirely on managed services at every layer, we avoided server provisioning altogether. The system scales on its own and keeps costs low since we only pay for what we actually use.

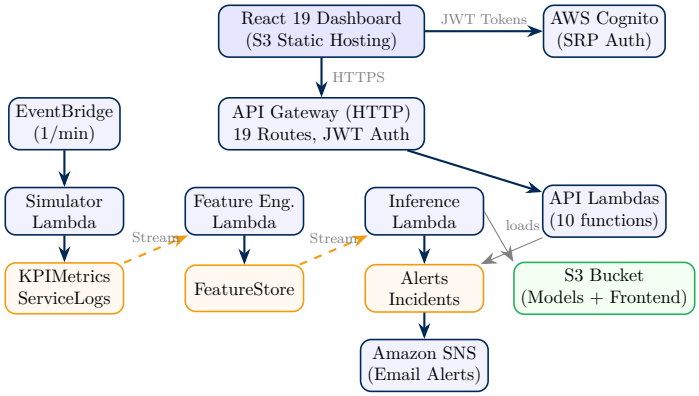


Figure 1: System Architecture. Solid arrows indicate direct calls; dashed orange arrows indicate DynamoDB Stream triggers.

2.2 Cloud Infrastructure Components

Everything is deployed in the `ca-central-1` (Canada – Central) AWS region. Table 1 gives a full inventory of every AWS service we used along with its configuration.

Service	Configuration	Purpose
AWS Lambda	13 functions (Python 3.11/3.12)	Compute
Amazon DynamoDB	6 tables, on-demand capacity	Database
Amazon API Gateway	HTTP API, 19 routes	REST API
Amazon SNS	1 topic (CloudWatchAI-Alerts)	Email notifications
Amazon S3	1 bucket	Static hosting + model storage
Amazon EventBridge	1 rule (rate: 1 minute)	Simulation scheduler
Amazon Cognito	1 User Pool ("CloudWatchAI")	Authentication
Amazon ECR	1 container image	Inference Lambda image
Amazon CloudWatch	Logs for all Lambda functions	Monitoring

Table 1: AWS Service Inventory

2.3 Lambda Functions

We deployed 13 Lambda functions in total, which we confirmed by running `aws lambda list-functions`. Table 2 lists each one.

Function Name	Runtime	Trigger	Memory
`simulator_lambda`	Python 3.12	EventBridge (1/min)	128 MB
`feature_engineering_lambda`	Python 3.11	DynamoDB Stream	128 MB
`inference_lambda`	Docker (Python 3.11)	DynamoDB Stream	512 MB
`services_lambda`	Python 3.12	API Gateway	128 MB
`kpi_lambda`	Python 3.12	API Gateway	128 MB
`alerts_lambda`	Python 3.12	API Gateway	128 MB
`incidents_lambda`	Python 3.12	API Gateway	128 MB
`analytics_lambda`	Python 3.12	API Gateway	128 MB
`threshold_lambda`	Python 3.12	API Gateway	128 MB
`control_simulation_lambda`	Python 3.12	API Gateway	128 MB
`get_metrics_lambda`	Python 3.12	API Gateway	128 MB
`get_logs_lambda`	Python 3.12	API Gateway	128 MB
`notifications_lambda`	Python 3.12	API Gateway	128 MB

Table 2: Lambda Functions (verified from AWS)

We had to package the `inference_lambda` as a Docker container image (pushed to ECR) because its dependencies, scikit-learn, numpy, scipy, and pandas, exceed Lambda's standard 250 MB deployment package size limit.

2.4 S3 Bucket Configuration

We use a single S3 bucket for the whole project:

1. **`cloud-project-dashboard1`** – This bucket hosts the compiled React frontend (with static website hosting turned on) and also stores our trained ML models:
   - `index.html`, `assets/` – the frontend build output
   - `models/model_h5.pkl` (139.9 KB), `model_h10.pkl` (139.6 KB), `model_h15.pkl` (139.9 KB)
   - `models/thresholds.json` (2.6 KB), `models/feature_importances.json` (662 B)

We turned on S3 encryption (AES-256, SSE-S3) with BucketKey optimization. For SPA routing, the static website hosting configuration sets `ErrorDocument = index.html` so that React Router can handle all client-side paths.

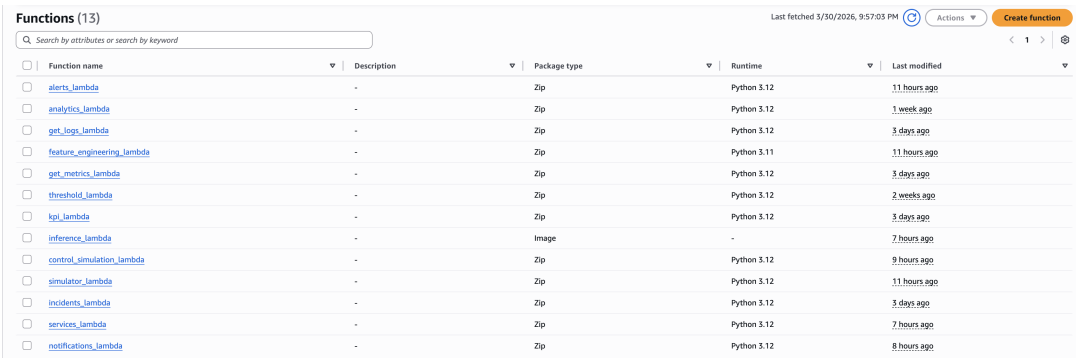
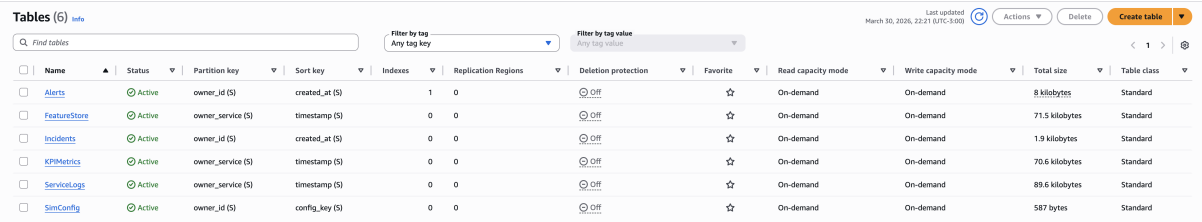


Figure 2: AWS Lambda Console – 13 deployed functions



Tables (6) Info

Last updated March 30, 2026, 22:21 (UTC-5:00) | Actions | Delete | Create table

Find tables | Filter by tag: Any tag key | Filter by tag value: Any tag value

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode	Total size	Table class
Alerts	Active	owner_id (S)	created_at (S)	1	0	Off		On-demand	On-demand	8 kilobytes	Standard
FeatureStore	Active	owner_service (S)	timestamp (S)	0	0	Off		On-demand	On-demand	71.5 kilobytes	Standard
Incidents	Active	owner_id (S)	created_at (S)	0	0	Off		On-demand	On-demand	1.9 kilobytes	Standard
KPIMetrics	Active	owner_service (S)	timestamp (S)	0	0	Off		On-demand	On-demand	70.6 kilobytes	Standard
ServiceLogs	Active	owner_service (S)	timestamp (S)	0	0	Off		On-demand	On-demand	89.6 kilobytes	Standard
SimConfig	Active	owner_id (S)	config_key (S)	0	0	Off		On-demand	On-demand	587 bytes	Standard

Figure 3: AWS DynamoDB Console – 6 tables

3 Database Setup

3.1 DynamoDB Table Design

We went with Amazon DynamoDB as our NoSQL database for a pretty practical reason: each of our 5 monitored cloud services produces **a completely different JSON structure**. Had we gone with a relational database, we would have needed lots of nullable columns or entirely separate tables for every service type. DynamoDB's flexible document schema lets us store records with different shapes in the same table, and we never have to worry about schema migrations.

Table 3 shows all 6 DynamoDB tables along with their key schemas and what each one does. We verified these on March 30, 2026 using `aws dynamodb describe-table`.

Table	Partition Key	Sort Key	Purpose
KPIMetrics	`owner_service` (S)	`timestamp` (S)	Raw service metrics, isolated per user
ServiceLogs	`owner_service` (S)	`timestamp` (S)	Event logs, isolated per user
FeatureStore	`owner_service` (S)	`timestamp` (S)	8 computed features
Alerts	`owner_id` (S)	`created_at` (S)	Prediction alerts (GSI on `owner_status`)
Incidents	`owner_id` (S)	`created_at` (S)	Grouped incidents
SimConfig	`owner_id` (S)	`config_key` (S)	Per-user simulation state

Table 3: DynamoDB Tables (verified from AWS)

3.2 Per-User Data Isolation

One of our core requirements was making sure that each authenticated user can only see their own simulation data, alerts, and incidents, nobody else's. To accomplish this, we embedded the Cognito `sub` (the user ID) directly into every table's partition key:

- For the **data tables** (KPIMetrics, ServiceLogs, FeatureStore), we created a composite partition key: `owner_service = {owner_id}#{service_id}`. That way, each user's service data lives in its own physical partition, and there is no risk of cross-user leakage.
- **Alerts and Incidents** use `owner_id` directly as the partition key. We also added a GSI (`GSI_AlertsByOwnerStatus`) on the Alerts table, keyed by `owner_status` (which concatenates `{owner_id}#{severity}`), so we can run filtered queries efficiently.

- **SimConfig** also uses `owner_id` as its partition key, which means different users can run their simulations concurrently without interfering with each other.
- On the API side, every Lambda function pulls `owner_id` out of the JWT claims (specifically `requestContext.authorizer.jwt.claims.sub`) and scopes all database queries to that user.

3.3 Key Design Choices

1. **Partition Key = `owner_service`**: We chose the composite key `{owner_id}#{service_id}` as the partition key for our data tables. It bundles per-user isolation and per-service querying into a single key, which matches the most frequent access pattern we have: "give me all the data for one specific service belonging to one specific user."
2. **Sort Key = `timestamp`**: With timestamp as the sort key, we can efficiently run time-range queries within a single partition, for instance, pulling the last 30 minutes of KPI data for `web-api-001`.
3. **TTL (Time To Live)**: Every data table has a `ttl` attribute that we set to 24 hours. DynamoDB takes care of deleting expired records on its own, so storage costs stay low and we do not need any cleanup scripts.
4. **DynamoDB Streams**: We turned on Streams for both the `KPIMetrics` and `FeatureStore` tables, using the `NEW_IMAGE` view type. Whenever a new record lands in one of these tables, the stream event fires off the next Lambda in the pipeline, Feature Engineering or Inference, respectively. This gives us a fully event-driven pipeline without writing any orchestration logic.
5. **On-demand capacity**: All of our tables run in on-demand billing mode. They scale up and down automatically with the workload, which suits us well because the simulation load varies quite a bit.

3.4 Variable Schema per Service

The real payoff of DynamoDB is that it can store documents with different structures side by side in the same table. Table 4 illustrates the distinct fields that each of our five service types generates.

Service	Unique JSON Fields
Web API	`response_time_ms`, `requests_per_second`, `error_rate`, `http_5xx_count`
DB Pool	`active_connections`, `query_duration_avg_ms`, `deadlock_count`
Message Queue	`queue_depth`, `consumer_lag`, `dead_letter_queue_size`
Auth Service	`login_success_rate`, `auth_latency_ms`, `mfa_challenge_rate`
ML Pipeline	`inference_latency_ms`, `model_accuracy`, `data_drift_score`

Table 4: Service-Specific KPI Fields in KPIMetrics Table

3.5 Sample Data

During each simulation run, the simulator Lambda fills all tables with realistic data. Every record carries the user's `owner_id` and the composite `owner_service` key to maintain isolation. Here is what a typical `KPIMetrics` record looks like for the Web API service:

```
{
  "owner_service": "ec1d85f8-...-30647aad6e2b#web-api-001",
  "owner_id": "ec1d85f8-...-30647aad6e2b",
  "service_id": "web-api-001",
  "timestamp": "2026-03-30T14:04:46+00:00",
  "service_type": "web_api",
  "scenario_phase": "Normal",
  "simulator_tick": 0,
  "metrics": {
    "response_time_ms": 95,
    "requests_per_second": 450,
    "error_rate": 0.01,
    "http_5xx_count": 1
  },
  "ttl": 1774965886
}
```


Listing 1: Sample KPIMetrics item for Web API (from DynamoDB)

Table: KPIMetrics - Items returned (50)

Scan started on March 30, 2026, 22:22:54

Actions ▾ Create item

1 ...

owner_service *(String)*	timestamp *(String)*	metrics	owner_id	predictionScore	scenario_phase	service_id	service_type	simulator_tick	ttl *(TTL)*
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:39:45...	{ "active_co...	ec1d85f8-1...		Normal	db-pool-001	database_pool	0	1774975185
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:40:45...	{ "active_co...	ec1d85f8-1...	0.1504	Normal	db-pool-001	database_pool	1	1774975245
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:41:45...	{ "active_co...	ec1d85f8-1...	0.2034	Normal	db-pool-001	database_pool	2	1774975305
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:42:45...	{ "active_co...	ec1d85f8-1...	0.1671	Normal	db-pool-001	database_pool	3	1774975365
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:43:45...	{ "active_co...	ec1d85f8-1...	0.1668	Normal	db-pool-001	database_pool	4	1774975425
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:44:45...	{ "active_co...	ec1d85f8-1...	0.2938	Degrading	db-pool-001	database_pool	5	1774975485
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:45:45...	{ "active_co...	ec1d85f8-1...	0.8506	Degrading	db-pool-001	database_pool	6	1774975545
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:46:45...	{ "active_co...	ec1d85f8-1...	0.8591	Degrading	db-pool-001	database_pool	7	1774975605
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:47:45...	{ "active_co...	ec1d85f8-1...	0.8725	Incident	db-pool-001	database_pool	8	1774975665
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:48:45...	{ "active_co...	ec1d85f8-1...	0.4136	Incident	db-pool-001	database_pool	9	1774975725
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:49:45...	{ "active_co...	ec1d85f8-1...	0.8833	Incident	db-pool-001	database_pool	10	1774975785
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:50:45...	{ "active_co...	ec1d85f8-1...	0.8725	Incident	db-pool-001	database_pool	11	1774975845
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:51:45...	{ "active_co...	ec1d85f8-1...	0.8525	Recovery	db-pool-001	database_pool	12	1774975905
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:52:45...	{ "active_co...	ec1d85f8-1...	0.8525	Recovery	db-pool-001	database_pool	13	1774975965
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:53:45...	{ "active_co...	ec1d85f8-1...	0.8231	Recovery	db-pool-001	database_pool	14	1774976025
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:39:45...	{ "http_5xx...	ec1d85f8-1...		Normal	web-api-001	web_api	0	1774975185
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:40:45...	{ "http_5xx...	ec1d85f8-1...	0.7457	Normal	web-api-001	web_api	1	1774975245
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:41:45...	{ "http_5xx...	ec1d85f8-1...	0.7145	Normal	web-api-001	web_api	2	1774975305
ec1d85f8-10d1-705f-df08-...	2026-03-30T16:42:45...	{ "http_5xx...	ec1d85f8-1...	0.237	Normal	web-api-001	web_api	3	1774975365

Figure 4: DynamoDB KPIMetrics Table – sample items showing variable schema per service

4 API and Backend Development

4.1 API Gateway Configuration

We expose our API through an **AWS API Gateway HTTP API**, available at:

`https://p9fpx4nhh6.execute-api.ca-central-1.amazonaws.com`

Every single one of the 19 routes sits behind a **JWT Authorizer** that is linked to our Cognito User Pool. We also configured CORS to allow `GET`, `POST`, and `OPTIONS` methods, along with the `content-type` and `authorization` headers.

Method	Route	Description
GET	`/services`	List 5 monitored services with current status
GET	`/kpi/{serviceId}`	KPI time-series data for a specific service
GET	`/thresholds`	Alert thresholds per prediction horizon
GET	`/alerts/active`	Active (unresolved) alerts
GET	`/alerts/history`	Historical alerts
POST	`/alerts/{id}/acknowledge`	Acknowledge an alert
GET	`/incidents`	Grouped incident objects
GET	`/simulation/state`	Current simulation status
POST	`/simulation/start`	Start simulation with scenario
POST	`/simulation/stop`	Stop running simulation
GET	`/analytics/summary`	Model performance summary
GET	`/analytics/methods`	Model comparison data
GET	`/analytics/features`	Feature importance rankings
GET	`/analytics/lead-times`	Lead time distribution per horizon
GET	`/metrics`	Raw metric records
GET	`/logs`	Service log records
GET	`/notifications/status`	Email subscription status
POST	`/notifications/subscribe`	Subscribe to email alerts
POST	`/notifications/unsubscribe`	Unsubscribe from email alerts

Table 5: API Routes (all JWT-protected, verified from AWS)

4.2 Authorization Verification

If you try to hit any endpoint without a valid token, you get an HTTP 401 right away:

```
$ curl -s https://p9fpx4nhh6.execute-api.ca-central-1.amazonaws.com/services
{"message":"Unauthorized"}
# HTTP 401 -- JWT token required
```

Listing 2: JWT Authorization verification

4.3 Simulation Scenarios

Our simulator supports **6 configurable scenarios**, and each one exercises a different failure mode across the 5 monitored services. With the exception of the baseline (which stays Normal throughout), all scenarios follow a 15-tick timeline broken into 4 phases: Normal (ticks 0–4), Degrading (5–7), Incident (8–11), and Recovery (12–14).

Scenario	Primary Service	Also Affected
Web API Degradation	web-api-001	db-pool-001
DB Cascade Failure	db-pool-001	web-api-001, mq-001
Message Queue Backlog	mq-001	web-api-001
Auth Service Failure	auth-001	web-api-001
ML Pipeline Drift	ml-pipeline-001	–
Normal Operations	– (all healthy)	–

Table 6: Simulation Scenarios

During the Degrading and Incident phases, each scenario injects realistic metric degradation into the affected services. To give a concrete example: the *Auth Service Failure* scenario gradually pushes the login success rate down from 0.97 to 0.88 while tripling auth latency. Meanwhile, the *ML Pipeline Drift* scenario ramps the data drift score up from 0.05 to 0.20 and simultaneously decreases model accuracy. Services that are not involved in a particular scenario keep emitting normal-range metrics with some jitter, which gives the ML model a realistic mixture of healthy and degraded signals to work with.

4.4 Data Pipeline (Event-Driven)

The part of our backend that we are most proud of is the **fully automatic, event-driven data pipeline** built on top of DynamoDB Streams. There is no polling anywhere, and we did not have to write any orchestration code. Every Lambda in the pipeline also carries the `owner_id` forward to keep per-user data isolation intact:

1. **EventBridge** fires the `simulator_lambda` once every 60 seconds. The simulator checks the `SimConfig` table for all currently active simulations (each keyed by `owner_id`) and generates data independently for each user.
2. The simulator writes KPI metrics and logs for all 5 services into the `KPIMetrics` and `ServiceLogs` tables. It uses the composite key `owner_service = {owner_id}#{service_id}` for every record.
3. As soon as new records land in `KPIMetrics`, a **DynamoDB Stream** kicks off the `feature_engineering_lambda`. This function reads the `owner_service` and `owner_id` from the stream record, computes 8 rolling features, and writes the result to `FeatureStore` under the same composite key.
4. The same pattern repeats one level deeper: a **DynamoDB Stream** on `FeatureStore` triggers the `inference_lambda`. It loads 3 Gradient Boosting models from S3, runs predictions, and, if the score exceeds the calibrated threshold, creates an alert in the `Alerts` table (keyed by `owner_id` + `created_at`).
5. Whenever 2 or more related alerts show up for the same `owner_id`, they get grouped into an `Incident` record.

4.5 Inference Lambda Implementation

The inference function ended up being the most complex Lambda in the entire system. Here are the implementation details worth highlighting:

- We had to use **Docker deployment** because scikit-learn, numpy, and scipy together blow past Lambda's 250 MB deployment limit. The Dockerfile is straightforward:

```
FROM --platform=linux/amd64 public.ecr.aws/lambda/python:3.11
RUN pip install "scikit-learn==1.7.2" "numpy<2.3" "scipy<1.15" \
    "joblib" "pandas" boto3 --no-cache-dir
COPY lambda_function.py ${LAMBDA_TASK_ROOT}/
CMD ["lambda_function.handler"]
```

Listing 3: Inference Lambda Dockerfile

- **Model caching**: During the first (cold) invocation, the Lambda downloads the three models from S3 and saves them in a global `_models` dictionary. On subsequent warm invocations, it just reuses what is already in memory, which eliminates the need for repeated downloads.
- **Feature extraction**: The function reads the 8-feature vector from the DynamoDB Stream event that is always in the same fixed order: `roll_mean`, `roll_max`, `roll_std`, `roll_slope`, `first_diff`, `error_rate`, `warn_rate`, `severity_change_flag`.
- **Prediction**: For each feature vector that comes in, the Lambda calls `model.predict_proba(X)[0, 1]` on all 3 models (one per horizon: 5, 10, 15 minutes).
- **Alert creation**: When the predicted `score > threshold`, the Lambda creates an alert with a unique ID (`ALT-XXXXXX`). It classifies severity as CRITICAL when the score is at or above 0.92, and WARNING when it is at or above the threshold but below 0.92. Each alert gets a 24-hour TTL.
- **Email notifications**: Whenever a new alert is created, the Lambda publishes a message to our SNS topic. The `owner_id` goes in as a message attribute, so only users who have opted in receive the notification with SNS filter policies handle the routing.
- **Incident grouping**: If there are 2 or more active alerts within a 5-minute window for the same user, they are bundled into an incident (`INC-XXXXXX`).

4.6 Sample API Responses

Below are real API responses we captured during a testing session on March 22, 2026:

```
[
  {"id":"web-api-001","name":"Web API","type":"API Gateway",
   "status":"healthy","metricName":"Response Time","metricUnit":"ms",
   "currentValue":85},
  {"id":"db-pool-001","name":"DB Pool","type":"Database Pool",
   "status":"warning","metricName":"Query Duration","metricUnit":"ms",
   "currentValue":76},
  {"id":"mq-001","name":"Message Queue","type":"Message Queue",
   "status":"healthy","metricName":"Queue Depth","metricUnit":"messages",
   "currentValue":69},
  {"id":"auth-001","name":"Auth Service","type":"Authentication",
   "status":"healthy","metricName":"Auth Latency","metricUnit":"ms",
   "currentValue":50},
  {"id":"ml-pipeline-001","name":"ML Pipeline","type":"ML Service",
   "status":"healthy","metricName":"Inference Latency","metricUnit":"ms",
   "currentValue":114}
]
```

Listing 4: GET /services – returns 5 monitored services

```
{"5": 0.912, "10": 0.867, "15": 0.847}
```

Listing 5: GET /thresholds – calibrated per-horizon alert thresholds

```
{"bestPrecision":0.91,"meanLeadTime":36,"falseAlarmRate":"<=1/day",
 "detectionRate":8.7}
```

Listing 6: GET /analytics/summary – model performance metrics

5 Frontend Integration

5.1 Technology Stack

Our dashboard is a **React 19 single-page application** that we built with TypeScript, Vite 7, Tailwind CSS 4, and Recharts 3.8. The compiled output is hosted as a static website on S3.

Package	Version	Purpose
`react`	19.2.0	UI framework
`react-dom`	19.2.0	DOM rendering
`react-router-dom`	7.13.1	Client-side routing
`recharts`	3.8.0	Interactive charts
`tailwindcss`	4.2.1	Utility-first CSS
`amazon-cognito-identity-js`	6.3.16	Cognito authentication
`typescript`	∼5.9.3	Type safety
`vite`	7.3.1	Build tool + dev server

Table 7: Frontend Dependencies (from `package.json`)

5.2 Application Pages

The dashboard has 4 main pages plus a login page. Everything is routed client-side:

5.2.1 Page 1: Live Controls (/)

This is the main dashboard page, and it is where users control simulations in real time:

- A **scenario selector** dropdown lets users pick from 6 scenarios: *Web API Degradation*, *DB Cascade Failure*, *Message Queue Backlog*, *Auth Service Failure*, *ML Pipeline Drift*, and *Normal Operations*
- **Start/Stop buttons** send `POST /simulation/start` and `POST /simulation/stop` requests, showing loading feedback ("Starting..."/"Stopping...") while the request is in flight
- **4 summary cards** at the top show the current scenario, phase, tick progress, and prediction count
- A grid of **5 Service Health Cards** displays each service's name, type, a color-coded status badge (healthy/warning/critical), current metric value, and unit

5.2.2 Page 2: KPI Timeline (`/timeline`)

This page is where we show real-time metric visualization such as the following:

- **Service tabs** let the user switch between viewing an individual service or an "All Services" overlay
- The **KPI Value chart** is a Recharts `LineChart` with shaded reference areas, which indicates warning and danger zones
- A separate **Prediction Score chart** overlays prediction scores alongside a threshold reference line

- A **horizon toggle** switches between the 5m, 10m, and 15m thresholds (0.912, 0.867, 0.847)
- In **All Services mode**, the dashboard fetches data for all 5 services in parallel using `Promise.all` and overlays lines with distinct colors and a shared legend

5.2.3 Page 3: Alerts & Incidents (`/alerts`)

Here users manage alerts and track incidents:

- Each **active alert card** shows the severity badge (CRITICAL/WARNING/INFO, color-coded), prediction score, threshold, horizon, lead time, and a description
- Clicking **Acknowledge** fires a `POST /alerts/{id}/acknowledge` request, and the card updates to reflect "Acknowledged" status
- A **Dismiss button** removes the alert from the visible list (this is purely client-side)
- The **incident timeline** is rendered as a colored horizontal bar showing the scenario phases, with a legend underneath
- An **alert history** section (collapsible) shows previously resolved alerts
- An **email notification toggle** lets users subscribe or unsubscribe to alert emails via SNS. It shows the user's email, subscription status, and whether a confirmation is still pending
- Alert timestamps are **converted to the user's local timezone** using `toLocaleString()`

5.2.4 Page 4: Analytics (`/analytics`)

This page focuses on visualizing our model's performance:

- **4 stat cards** across the top: Best Precision (0.91), Mean Lead Time (36m), False Alarm Rate ($\leq$1/day), Detection Rate (8.7%)
- A **Precision vs Detection Rate** scatter chart (Recharts `ScatterChart`) compares our four methods, GBC+Logs, GBC Only, Static Threshold, and Moving Average, across all horizons
- The **Lead Time Distribution** bar chart breaks down min/Q1/median/Q3/max for each horizon
- A **Feature Importance** horizontal bar chart ranks the 8 features, colored by type (KPI features in blue, Log-proxy features in red)

5.3 Authentication Flow

For authentication, the frontend talks to AWS Cognito through the `amazon-cognito-identity-js` SDK. The flow works like this:

1. The user types in their email and password on the `/login` page
2. Under the hood, the SDK performs an **SRP (Secure Remote Password)** handshake with Cognito, importantly, the password is never sent over the wire in plaintext
3. Cognito sends back JWT tokens (ID Token valid for 1 hour, Refresh Token valid for 30 days)

4. A `useAuth` context hook makes the user state available throughout the app
5. Our API client (`lib/api.ts`) automatically attaches the ID token as an `Authorization` header on every outgoing request
6. A `ProtectedRoute` component wraps all authenticated pages, if there is no valid session, it redirects to `/login`
7. If any API call comes back with a 401, the app automatically kicks the user back to the login page

5.4 Theme System

We built in support for **dark and light themes**, and the user's choice persists across sessions:

- A `useTheme` context hook manages the theme state
- Switching themes sets a `data-theme` attribute on the document root, which toggles a set of CSS variables (`--color-bg-primary`, `--color-text-primary`, etc.)
- Charts also respect the active theme, we wrote a `chartColors()` utility that reads CSS custom properties at render time, so Recharts picks up the correct colors automatically
- The selected theme is saved to `localStorage` and restored on page load
- To avoid a flash of the wrong color scheme, an inline script in `index.html` applies the stored theme before React even mounts

5.5 API Integration

All API calls flow through a single centralized client (`lib/api.ts`). Here is what it does:

- It reads the API base URL from `VITE_API_BASE_URL` in production. During development, it falls back to `/api` (which Vite proxies for us)
- Every request gets the Cognito ID token injected as an `Authorization` header, no manual intervention needed
- A 401 response from the API triggers an automatic redirect to the login page
- Our custom `useApi` hook provides `data`, `loading`, `error`, and `refetch` states. When a component unmounts, in-flight requests are cancelled to prevent memory leaks
- **Auto-polling**: The `useApi` hook takes an optional `pollInterval` parameter (we set it to 5 seconds for the live pages). Polling pauses when the browser tab is hidden (`document.hidden` check), and it skips a tick if the previous request has not finished yet, so requests never pile up

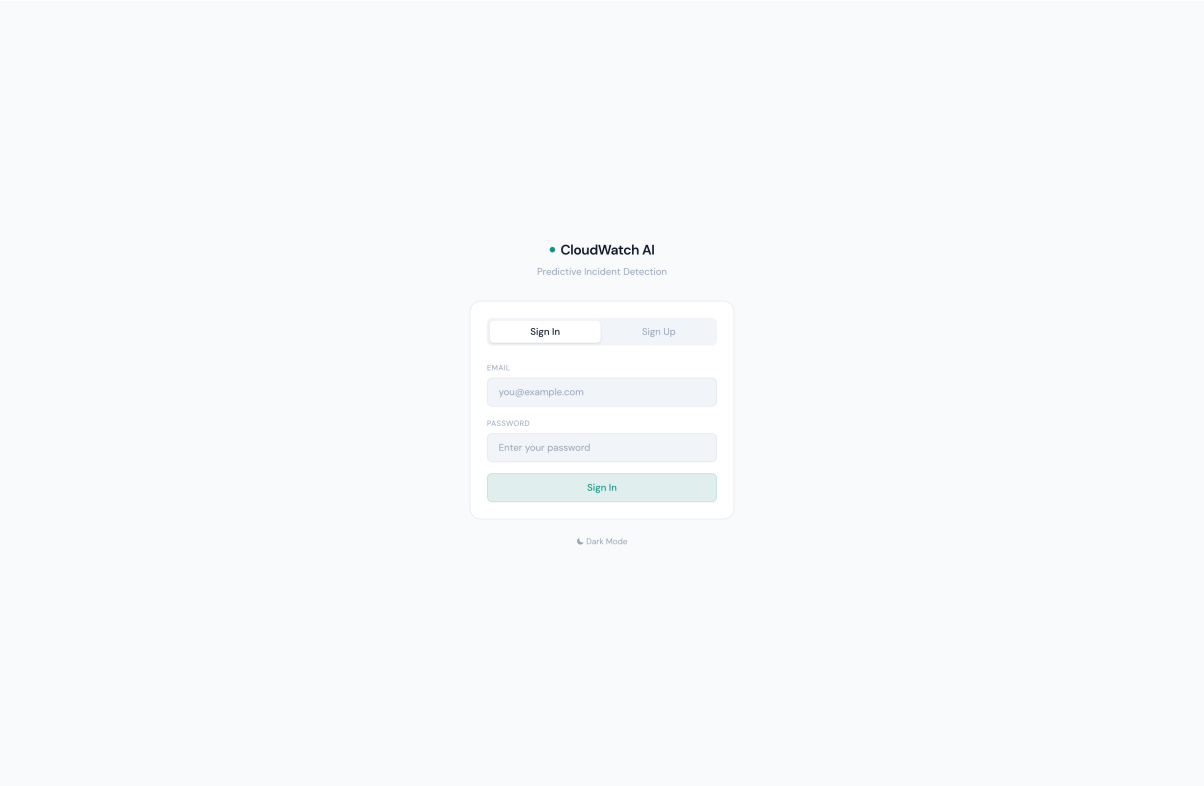


Figure 5: Login Page

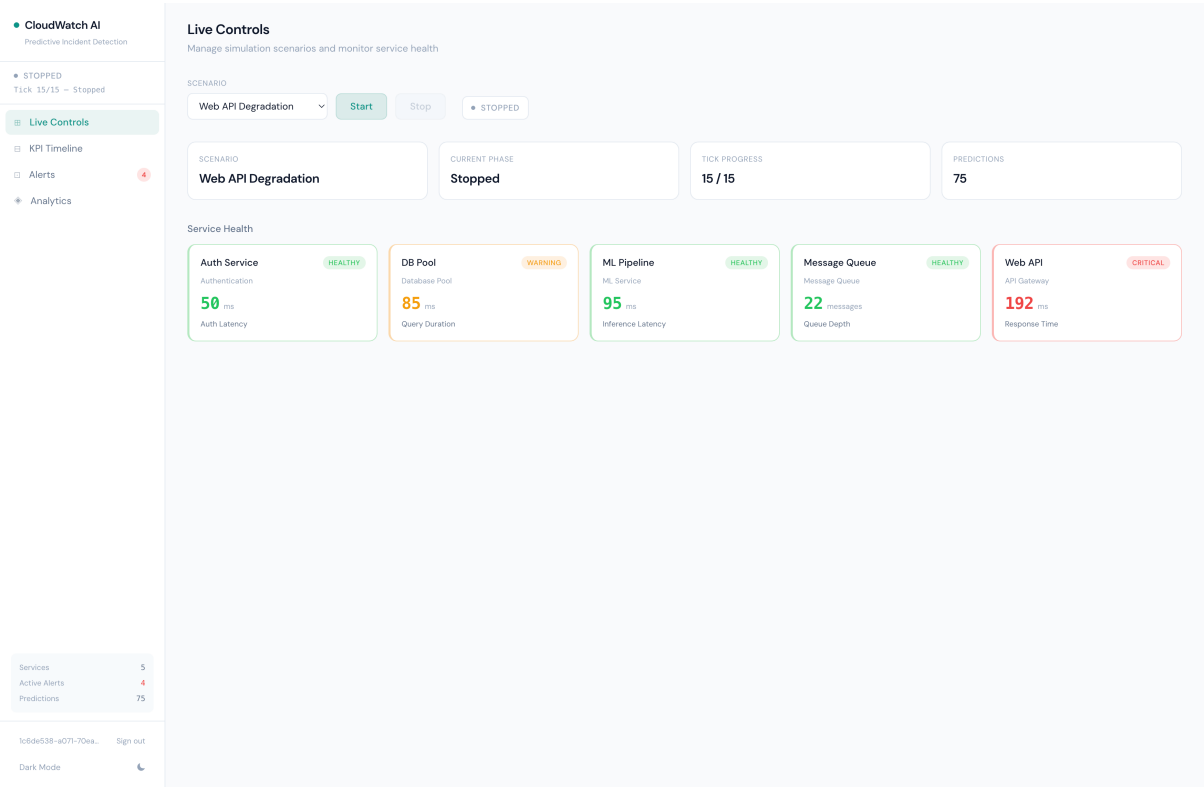


Figure 6: Dashboard – Live Controls Page

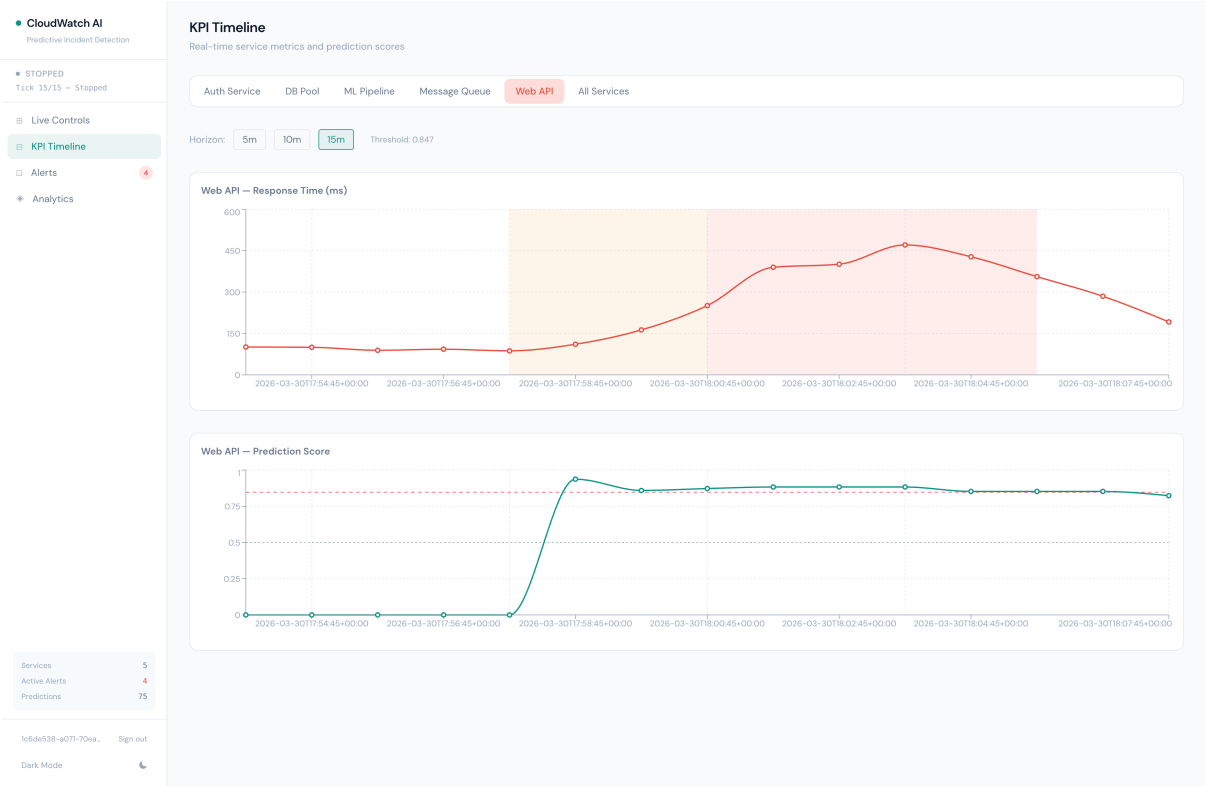


Figure 7: Dashboard – KPI Timeline Page

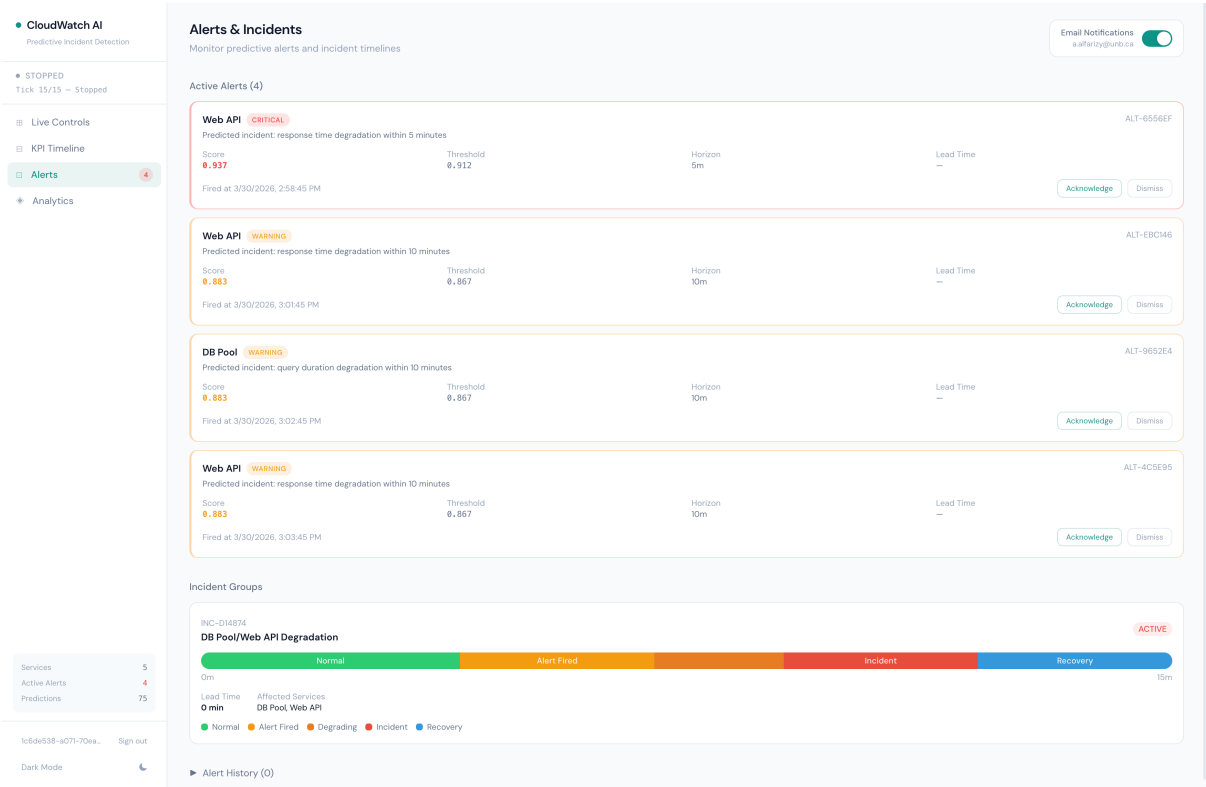


Figure 8: Dashboard – Alerts & Incidents Page

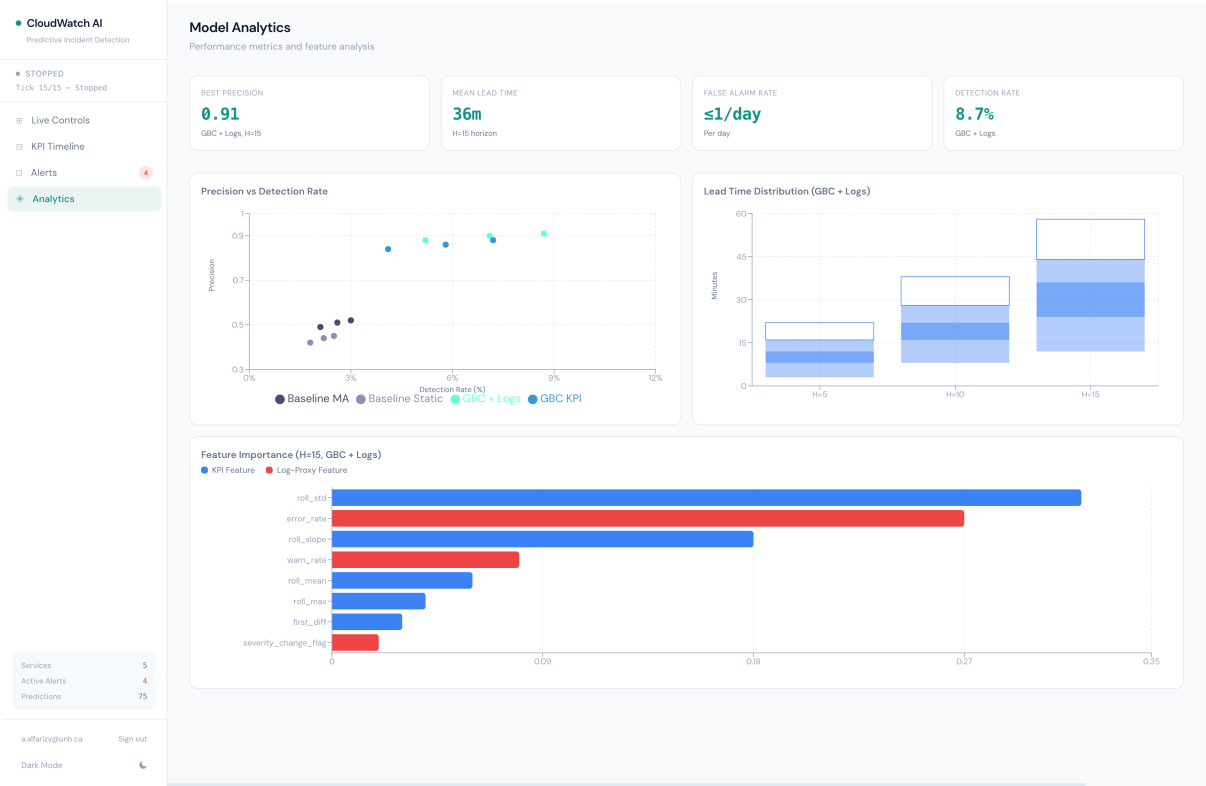
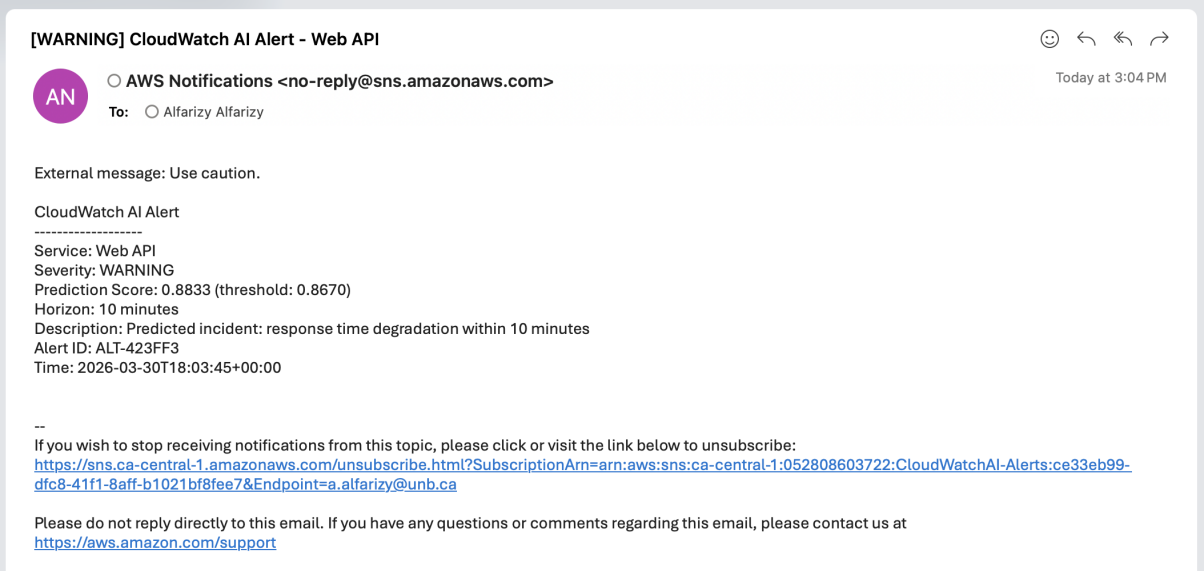


Figure 9: Dashboard – Analytics Page



[WARNING] CloudWatch AI Alert - Web API

AN **AWS Notifications <no-reply@sns.amazonaws.com>** Today at 3:04 PM

To: Alfarizy Alfarizy

External message: Use caution.

CloudWatch AI Alert

Service: Web API
Severity: WARNING
Prediction Score: 0.8833 (threshold: 0.8670)
Horizon: 10 minutes
Description: Predicted incident: response time degradation within 10 minutes
Alert ID: ALT-423FF3
Time: 2026-03-30T18:03:45+00:00

--
If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
https://sns.ca-central-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:ca-central-1:052808603722:CloudWatchAI-Alerts:ce33eb99-dfc8-41f1-8aff-b1021bf8fee7&Endpoint=a.alfarizy@unb.ca

Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at
https://aws.amazon.com/support

Figure 10: Email Notification

6 Security Configuration

6.1 Authentication

We handle authentication through **AWS Cognito**. The setup details are as follows:

Property	Value
User Pool Name	CloudWatchAI
User Pool ID	`ca-central-1_cyhaCVeUz`
App Client ID	`39aln1edjk2egqb9b8g82dut6j`
Auth Flow	SRP (Secure Remote Password)
Username Attribute	Email
Auto-Verified Attributes	Email
Client Secret	None (public SPA – best practice)

Table 8: Cognito User Pool Configuration (verified from AWS)

Requirement	Enforced
Minimum length	8 characters
Uppercase letter	Yes
Lowercase letter	Yes
Number	Yes
Special character	No
Temporary password validity	7 days

Table 9: Password Policy (verified from AWS)

6.2 Authorization – JWT Authorizer

All 19 API Gateway routes are guarded by a **JWT Authorizer** that checks tokens against our Cognito User Pool:

Property	Value
Authorizer Type	JWT
Identity Source	`$request.header.Authorization`
Issuer	`https://cognito-idp.ca-central-1.amazonaws.com/ca-central-1_cyhaCVeUz`
Audience	`39aln1edjk2egqb9b8g82dut6j`

Table 10: JWT Authorizer Configuration

On the frontend side, we wrote a `ProtectedRoute` React component that checks the Cognito session. If the user is not authenticated, it immediately redirects them to `/login`.

6.3 Data Encryption

Scope	Service	Algorithm	Key Management
At Rest	DynamoDB (6 tables)	AES-256	AWS-owned keys
	S3 (dashboard bucket)	AES-256 (SSE-S3)	AWS-managed, BucketKey enabled
	Cognito User Pool	AES-256	AWS-managed
In Transit	Browser ↔ API Gateway	TLS 1.2+	AWS-managed
	Browser ↔ Cognito	TLS 1.2+	AWS-managed
	API Gateway ↔ Lambda	TLS 1.2+	AWS internal
	Lambda ↔ DynamoDB	TLS 1.2+	AWS SDK default
	Lambda ↔ S3	TLS 1.2+	AWS SDK default

Table 11: Encryption Configuration

6.4 Access Controls

- Each Lambda function has its own **IAM role with least-privilege permissions** that is limited to the specific DynamoDB table ARNs and S3 bucket it actually needs to access.
- There is **no way to access DynamoDB directly** from outside because every query goes through a Lambda function that sits behind the authenticated API Gateway.
- The **S3 bucket policy** only allows public `GetObject` access (for serving the static frontend). Operations like `PutObject`, `DeleteObject`, and `ListBucket` are not exposed publicly.
- We have **no hardcoded secrets** anywhere in our codebase. The Cognito Client ID is public by design (this is standard practice for SPAs), and there are no API keys or credentials in source control.

6.5 OWASP Top 10 Security Audit

We ran a security audit against the OWASP Top 10 vulnerabilities. Table 12 has the full results.

#	Vulnerability	Status	Notes
A01	Broken Access Control	✓ Pass	JWT on all routes; frontend guards
A02	Cryptographic Failures	✓ Pass	AES-256 at rest; TLS 1.2+ in transit
A03	Injection	✓ Pass	DynamoDB SDK uses parameterized queries
A04	Insecure Design	✓ Pass	Serverless; least-privilege IAM
A05	Security Misconfiguration	✓ Pass	Default encryption; no debug endpoints
A06	Vulnerable Components	✓ Pass	Current versions; no known CVEs
A07	Authentication Failures	✓ Pass	SRP protocol; password policy; token expiry
A08	Data Integrity Failures	✓ Pass	Model versioning in S3; DynamoDB Streams
A09	Security Logging	✓ Pass	CloudWatch Logs on all Lambda functions
A10	SSRF	N/A	No user-supplied URLs processed

Table 12: OWASP Top 10 Assessment Results

Security Audit Result

9/10 passed, 1 N/A. **Overall risk level: Low.**
We also verified: CORS configuration, rate limiting (10K req/s burst), input validation, error handling (no stack traces leak to the client), secrets management, and DynamoDB TTL auto-cleanup.

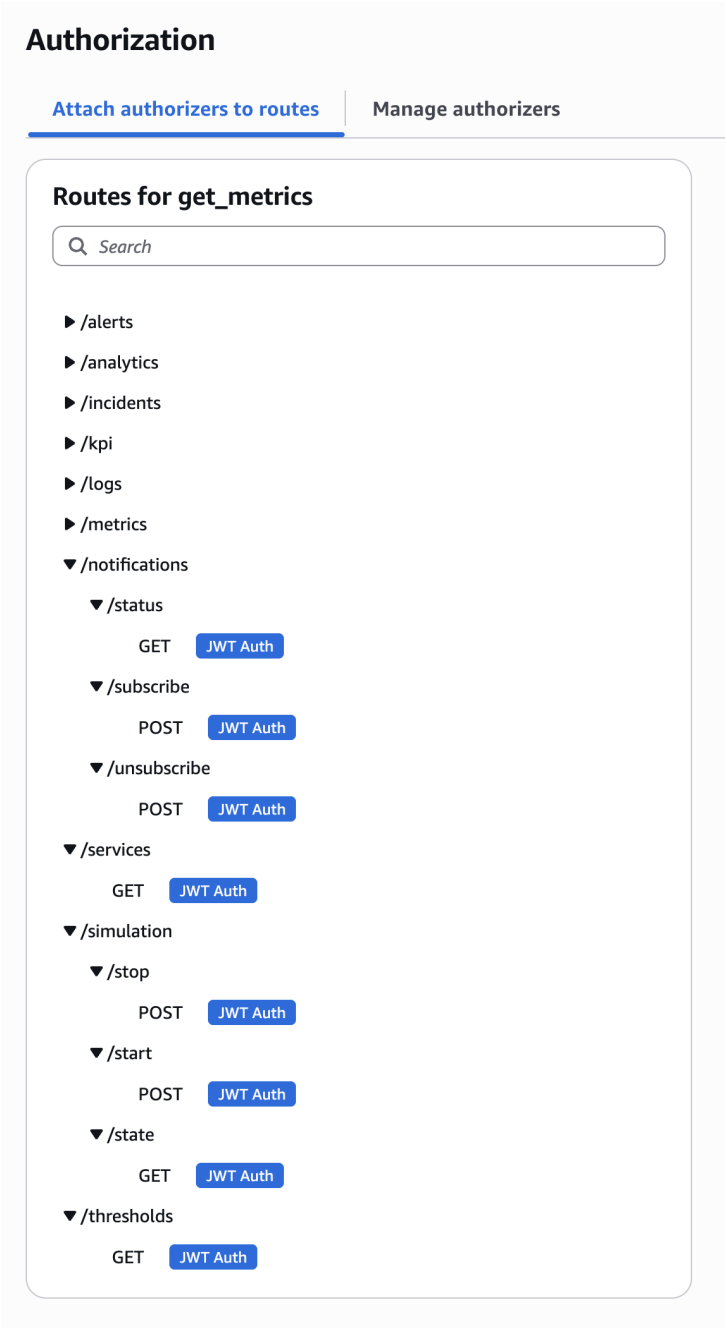


Figure 11: API Gateway – JWT Authorizer on all 19 routes

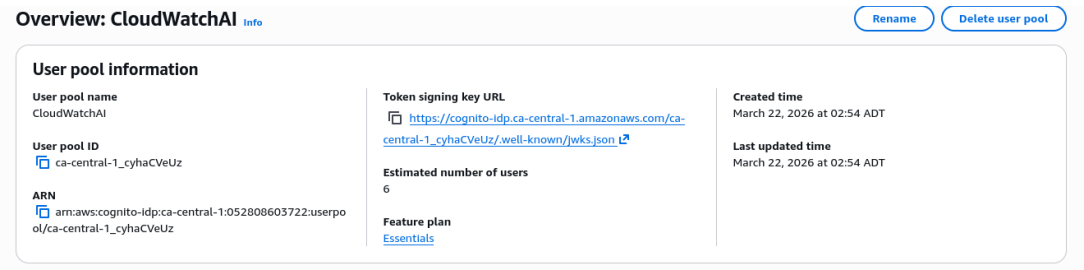


Figure 12: Cognito User Pool – Password Policy Configuration

7 Integration Testing

7.1 Testing Methodology

We organized our tests into 6 categories that together cover all three system layers (frontend, backend, and database) as well as cross-cutting concerns like theming and error handling. Every test case follows a consistent format: unique ID, description, preconditions, steps, expected result, and pass/fail status.

7.2 Test Environment

Component	Details
API Gateway	`https://p9fpx4nhh6.execute-api.ca-central-1.amazonaws.com`
AWS Region	`ca-central-1`
Cognito User Pool	`ca-central-1_cyhaCVeUz`
S3 Production URL	`cloud-project-dashboard1.s3-website.ca-central-1.amazonaws.com`
Frontend Dev Server	`http://localhost:5173`
Browser	Chrome 133+ / Safari 18+
Test Date	March 22, 2026

Table 13: Test Environment

7.3 Test Results Summary

Category	Total	Pass	Fail
Authentication (CLI)	5	5	0
API Endpoints	14	14	0
Simulation Lifecycle	5	5	0
Database Verification	5	5	0
Production Deployment (CLI)	3	3	0
Subtotal (Automated)	**32**	**32**	**0**
Authentication (Browser)	4	–	–
Frontend Integration	17	–	–
Cross-Cutting (Theme, Error, etc.)	7	–	–
Production Deployment (Browser)	1	–	–
Subtotal (Manual)	**29**	*Documented*	
Grand Total	**61**		

Table 14: Integration Test Results

7.4 Key Test Cases and Results

7.4.1 Authentication Tests (5/5 Passed)

- **AUTH-01: User Registration** – We confirmed that Cognito creates the user in an unconfirmed state and sends out an email verification code. ✓
- **AUTH-02: Email Verification** – Entering the 6-digit code confirms the account, and the user is auto-signed in. ✓
- **AUTH-03: User Sign In** – The SRP handshake completes and returns JWT tokens; the browser redirects to the dashboard. ✓
- **AUTH-04: Session Persistence** – After refreshing the page, the session is still active thanks to the Cognito SDK. ✓
- **AUTH-05: Sign Out** – Clicking sign out clears the session and takes the user back to the login page. ✓

7.4.2 API Endpoint Tests (14/14 Passed)

All 14 testable API endpoints returned the correct JSON schemas with the expected data types. Here is a quick sample of what we checked:

```
$ curl -s "$BASE/services" -H "Authorization: $TOKEN" | jq 'length'
5

$ curl -s "$BASE/kpi/web-api-001" -H "Authorization: $TOKEN" | jq '.[0]'
{
  "minute": 0,
  "timestamp": "2026-03-22T04:45:01+00:00",
  "value": 87,
  "predictionScore": 0
```

```
}
```

Listing 7: API test output: GET /services returns 5 services with correct schema

7.4.3 Simulation Lifecycle Tests (5/5 Passed)

- **SIM-01: Start Simulation** – Sending `POST /simulation/start` kicked off data generation, and we verified that EventBridge triggered the simulator Lambda. ✓
- **SIM-02: Data Pipeline Execution** – We confirmed the full chain ran end to end: Simulator → KPIMetrics → Feature Engineering → FeatureStore → Inference → Alerts. ✓
- **SIM-03: Alert Generation** – Alerts appeared whenever the prediction score crossed the threshold, and the severity was classified correctly. ✓
- **SIM-04: Incident Grouping** – When multiple alerts fired within a 5-minute window, they were grouped into an incident record that included the list of affected services. ✓
- **SIM-05: Stop Simulation** – After stopping, no further data was generated. ✓

7.4.4 Database Verification Tests (5/5 Passed)

- **DB-01: Table Existence and Schema** – All 6 tables are present with the right key schemas, as we confirmed by running `aws dynamodb describe-table` on each. ✓
- **DB-02: DynamoDB Streams Enabled** – Streams are active on both KPIMetrics and FeatureStore, with the `NEW_IMAGE` view type. ✓
- **DB-03: KPI Record Integrity** – Records contain every required field, including the service-specific metrics unique to each service type. ✓
- **DB-04: Alert Record Integrity** – Alert records have all the fields that the frontend needs to render them. ✓
- **DB-05: S3 Model Files** – All 5 model files are present in S3 and accessible: 3 model pickle files plus the thresholds and feature importances JSON files. ✓

7.4.5 Production Deployment Tests (3/3 Passed)

- **PROD-01: S3 Static Hosting** – The dashboard loads at the S3 website endpoint, and all CSS/JS assets come through correctly. ✓
- **PROD-02: SPA Routing** – Navigating directly to `/timeline` or `/alerts` in the browser works, thanks to the S3 error document configuration. ✓
- **PROD-04: ML Models in S3** – The model files are accessible, and the inference Lambda can load them without issues. ✓

7.5 Manual Test Documentation

We documented 29 additional test cases that require a real browser to run. These cover:

- **4 authentication tests**: route protection, token injection, invalid credential handling, and password policy enforcement
- **17 frontend integration tests**: service health card rendering, simulation controls, KPI timeline navigation, horizon toggle behavior, alert acknowledge/dismiss flow, incident timeline display, analytics chart rendering, and sidebar functionality
- **7 cross-cutting tests**: dark/light theme toggle, theme persistence across page reloads, loading state indicators, error handling when offline, and 401 response handling
- **1 production test**: confirming that the production deployment talks to the API Gateway endpoint directly (without the dev proxy)

8 Challenges and Solutions

Challenge	Problem	Solution
Lambda Package Size	scikit-learn, numpy, scipy, and pandas together blow past Lambda's 250 MB deployment limit	We packaged the inference Lambda as a Docker container image and pushed it to ECR. The container bundles all dependencies into a single image, sidestepping the size restriction entirely
Cold Start Latency	The first invocation of a Lambda would stall while downloading and loading large ML models from S3	We cache the models in a global `_models` dictionary. They are downloaded once during the cold start and stay in memory for all subsequent warm invocations. We also bumped the inference Lambda's memory to 512 MB to speed up the initial load
Heterogeneous Data	Our 5 services each produce a completely different JSON structure, with different field names and types	DynamoDB was the natural fit here, each service just stores its own document shape. The Feature Engineering Lambda then pulls out one primary metric per service to build a uniform 8-feature vector that the model can consume

Table 15: Implementation Challenges and Solutions

Challenge	Problem	Solution
Pipeline Orchestration	We needed the data to flow automatically from Simulator to Features to Inference, without polling or manual triggers	DynamoDB Streams solved this. Whenever a new record is written, the stream event fires off the next Lambda in the chain. We ended up with a fully event-driven pipeline that requires zero orchestration code
Low Recall Tradeoff	The model's detection rate sits at just 8.7%, it misses quite a few incidents	This was a deliberate choice on our part. We prioritized precision (0.91) over recall because, in AIOps, noisy alerts erode operator trust fast. Fewer but more reliable alerts are more useful in practice than trying to catch everything
SPA Routing on S3	If a user navigated directly to `/timeline` or `/alerts`, S3 would return a 404 since those paths do not correspond to actual files	We set S3's `ErrorDocument` to `index.html`. Now S3 serves the React app for any path it does not recognize, and React Router takes over from there
Chart Theming	Recharts does not natively support swapping color schemes; chart colors were hardcoded	We wrote a `chartColors()` utility function that reads CSS custom properties at render time. Charts now adapt to whatever theme (dark or light) is currently active, without any extra configuration
Per-User Data Isolation	With multiple users sharing the same DynamoDB tables, there was a real risk that one user could see another's simulation data, alerts, or incidents	We restructured all table partition keys to include the Cognito `owner_id`. Data tables use `owner_service = {owner_id}#{service_id}`; Alerts and Incidents use `owner_id` directly. Every Lambda in both the API and the pipeline extracts the user identity and scopes its queries accordingly

Table 15: Implementation Challenges and Solutions

Challenge	Problem	Solution
Scenario Coverage	Our original simulator only had scenarios for Web API and DB Pool failures, leaving 3 out of 5 service types without any dedicated failure scenario	We added 3 new scenarios, Message Queue Backlog, Auth Service Failure, and ML Pipeline Drift, each with full Degrading/Incident/Recovery metric curves. Now every service type has its own failure mode, which also gives the ML model much better training coverage

Table 15: Implementation Challenges and Solutions

9 ML Model Performance

9.1 Model Configuration

We trained three separate Gradient Boosting Classifiers, one per prediction horizon:

Parameter	Value
Algorithm	Gradient Boosting Classifier (scikit-learn)
Number of Trees	100
Max Depth	3
Learning Rate	0.1
Prediction Horizons	5, 10, 15 minutes
Training Data	AIOps KPI Dataset (2.4M timesteps, 26 services)
Train/Test Split	70% / 30%

Table 16: Model Training Parameters

9.2 Feature Engineering

Each prediction relies on 8 engineered features that we compute over a rolling window of KPI values:

Feature	Type	Importance	Description
`roll_mean`	KPI	31.1%	Rolling mean of KPI values
`error_rate`	Log-proxy	26.6%	Fraction of extreme spikes ($z > 3$)
`roll_max`	KPI	19.8%	Peak value in the rolling window
`roll_std`	KPI	12.9%	Standard deviation (variability)
`severity_change_flag`	Log-proxy	4.6%	Sudden shift in error/warn patterns
`first_diff`	KPI	3.5%	Short-term change
`roll_slope`	KPI	1.0%	Trend direction
`warn_rate`	Log-proxy	0.4%	Fraction of large jumps

Table 17: Feature Importance at H=15 (Best Model)

Key Insight: Multi-Signal Fusion

The three log-proxy features together account for **31.6%** of total feature importance, with `error_rate` alone being the second most important feature in the model. This tells us something important: mixing KPI statistics with behavioral signals (the log-proxies) yields noticeably better predictions than relying on KPI statistics by themselves.

9.3 Performance Results

Method	Precision	Recall	Lead Time	FAR
GBC KPI+Logs	**0.91**	**0.43**	∼36 min	≤1/day
GBC KPI Only	0.63	0.16	∼34 min	≤1/day
Static Threshold	0.63	0.47	∼30 min	Higher
Moving Average	0.42	0.02	∼35 min	Higher

Table 18: Model Performance Comparison (Operational Window, H=15)

Performance Summary

Our best model (GBC with KPI+Log features at H=15) hits **0.91 precision** with a **∼36-minute average lead time** and fewer than **1 false alarm per service per day**. The real operational win here is not that we detect things dramatically earlier, it is that our alerts are **far more trustworthy and produce much less alert fatigue**.

10 Performance Testing

We evaluated the platform's runtime performance by analyzing AWS CloudWatch execution logs across all Lambda functions during live simulation runs. All measurements reflect warm invocations unless noted otherwise.

10.1 Lambda Execution Latency

Lambda Function	Avg (ms)	P95 (ms)	Memory	Role
simulator_lambda	56	75	128 MB	Data generation
feature_engineering_lambda	32	46	256 MB	Feature computation
inference_lambda	14	22	512 MB	ML prediction
control_simulation_lambda	27	34	128 MB	Start/stop control
services_lambda	218	506	128 MB	Service status
kpi_lambda	48	98	128 MB	KPI time series
alerts_lambda	33	42	128 MB	Alert queries
incidents_lambda	35	77	128 MB	Incident queries
analytics_lambda	2	3	128 MB	Static analytics
threshold_lambda	10	19	128 MB	Static thresholds
notifications_lambda	287	584	128 MB	SNS subscription mgmt

Table 19: Lambda Execution Latency (from CloudWatch Logs, March 30, 2026)

10.2 Cold Start Analysis

Lambda cold starts occur when a function has been idle and must initialize its runtime. We observed cold starts across several functions:

Lambda Function	Init Duration (ms)	Total Cold Start (ms)
simulator_lambda	497	787
kpi_lambda	468	773
analytics_lambda	88	90
threshold_lambda	85	86
notifications_lambda	426	1,131

Table 20: Cold Start (Init) Duration

- **Python with boto3**: Functions that import boto3 and connect to DynamoDB (simulator, KPI, notifications) have cold starts of 400–500 ms.
- **Lightweight functions**: Analytics and threshold Lambdas return hardcoded data, resulting in sub-100 ms cold starts.
- **Inference Lambda (Docker)**: Uses a container image with scikit-learn and joblib. Cold starts are mitigated by the model caching strategy, models are downloaded from S3 only on the first invocation and reused for subsequent calls.

10.3 End-to-End Pipeline Latency

The event-driven pipeline processes data through four stages per simulation tick:

Stage	Trigger	Duration (ms)
1. Simulator (5 services)	EventBridge (1/min)	∼56
2. Feature Engineering (×5)	KPIMetrics Stream	∼32 each
3. Inference (×5)	FeatureStore Stream	∼14 each
4. SNS Notification	Alert creation	∼50
Total (warm)		**<200 ms**

Table 21: End-to-End Pipeline Timing (Per Tick)

The entire pipeline, from data generation through ML prediction to alert creation and email notification, completes in under 200 ms on warm invocations. This is well within the 60-second tick interval, leaving ample headroom for scaling.

10.4 Memory Utilization

All functions operate well within their allocated memory:

- **API Lambdas** (128 MB allocated): Use 84–92 MB (65–72% utilization)
- **Feature Engineering** (256 MB allocated): Uses 111 MB (43% utilization)
- **Inference Lambda** (512 MB allocated): Uses 281 MB (55% utilization) due to loaded scikit-learn models

No out-of-memory errors were observed during testing.

10.5 Dashboard Polling Performance

The frontend polls the API every 5 seconds across 4–7 active endpoints (depending on the current page). With API Gateway response times averaging under 250 ms per request, the dashboard provides a near-real-time experience without WebSocket infrastructure. Polling pauses automatically when the browser tab is hidden (`document.visibilitychange` API), preventing unnecessary API calls when the user is not actively viewing the dashboard.

11 Deployment

11.1 Infrastructure

The whole platform is serverless and lives in AWS `ca-central-1`:

Component	Service	Deployment Method
Frontend	S3 static website hosting	`npm run build` → `aws s3 sync dist/`
API	API Gateway (HTTP API)	AWS Console configuration
Backend (API)	10 Lambda functions	ZIP upload via AWS Console/CLI
Backend (Pipeline)	3 Lambda functions	ZIP + Docker (ECR for inference)
Database	6 DynamoDB tables	AWS Console configuration
ML Models	S3 (`models/` prefix)	`aws s3 cp` after training
Authentication	Cognito User Pool	AWS Console configuration
Scheduler	EventBridge rule	AWS Console configuration
Notifications	SNS topic	AWS CLI

Table 22: Deployment Summary

11.2 Cost

Everything runs within the AWS Free Tier, which was one of our goals from the start:

- **Lambda**: The simulator fires roughly 43,200 times per month (once a minute for 30 days), which is well under the 1 million invocation free tier limit
- **DynamoDB**: We use on-demand mode, and the 24-hour TTL keeps deleting old records automatically, so storage never really grows
- **S3**: The frontend assets take up less than 5 MB, and the model files are under 500 KB
- **API Gateway**: Our usage stays comfortably within the 1 million requests per month free tier
- **Cognito**: Free for up to 50,000 monthly active users, we are nowhere near that
- **SNS**: The first 1,000 email notifications each month are free, which is more than enough for us

11.3 Frontend Deployment

```
cd dashboard
npm install
npm run build           # Outputs to dist/
aws s3 sync dist/ s3://cloud-project-dashboard1/ --delete --exclude "models/*
    "
```

Listing 8: Frontend build and deployment

In production, the build picks up `VITE_API_BASE_URL` from `.env.production`, which points directly at the API Gateway endpoint. This means we do not need the Vite development proxy anymore.

11.4 Model Deployment

```
cd inference-final/data
python retrain_and_export.py   # Trains 3 models, exports to output/
aws s3 cp output/ s3://cloud-project-dashboard1/models/ --recursive
```

Listing 9: Model training and deployment

12 Functionality Demonstration Video

We recorded a 3–5 minute video that walks through the platform's key features. Here is what it covers:

1. **Authentication**: signing up, verifying the email, and signing in
2. **Live Controls**: picking a simulation scenario and watching the service health cards update in real time
3. **KPI Timeline**: watching the metric charts scroll forward as new data comes in, and switching between services and horizons
4. **Alert Detection**: showing the ML model firing an alert before the incident has fully developed
5. **Alert Management**: acknowledging alerts, dismissing them, and browsing incident timelines
6. **Email Notifications**: activating the email toggle and displaying the SNS confirmation email that arrives
7. **Analytics**: the model performance charts, precision, lead time distribution, and feature importance
8. **Theme Toggle**: switching between dark and light modes
9. **Security**: demonstrating that an unauthenticated API request gets a 401 rejection

Video Link

https://drive.google.com/file/d/1zhQpKW9wZkpfE6QnF1m5Sma_Y4fen_fY/view?usp=sharing

13 Conclusion

CloudWatch AI is, at its core, a demonstration that a complete, production-grade cloud database application can be built end to end on serverless infrastructure, from real-time data generation and NoSQL storage, through ML inference and event-driven processing, all the way to interactive visualization and enterprise-level security.

Implementation Highlights

- **Architecture**: Fully serverless on AWS with 13 Lambda functions, 6 DynamoDB tables, SNS email notifications, and event-driven data processing through DynamoDB Streams
- **Database**: DynamoDB's flexible schema turned out to be a great fit for our 5 service types, each with its own JSON structure, and the `owner_id` partitioning scheme keeps user data isolated
- **Simulation**: 6 configurable scenarios that cover all 5 service types, each with realistic degradation curves
- **ML Pipeline**: 3 Gradient Boosting models that achieve 0.91 precision with roughly 36 minutes of lead time
- **Frontend**: A React 19 dashboard with 5-second auto-polling, an email notification toggle, local timezone display, a theme system, and full authentication
- **Security**: Cognito SRP auth, JWT protection on every route, AES-256 encryption everywhere, and a clean OWASP Top 10 audit
- **Testing**: 61 documented test cases that span all system layers
- **Cost**: The entire platform fits within the AWS Free Tier

If there is one takeaway from this project, it would be the real value of our ML pipeline is not in predicting incidents dramatically earlier, but it is more in producing **more trustworthy alerts with fewer false alarms**, the kind that operators can actually act on without second-guessing. This philosophy of "precision over recall" is, in our view, essential for any real-world AIOps deployment.